

Weather Management System

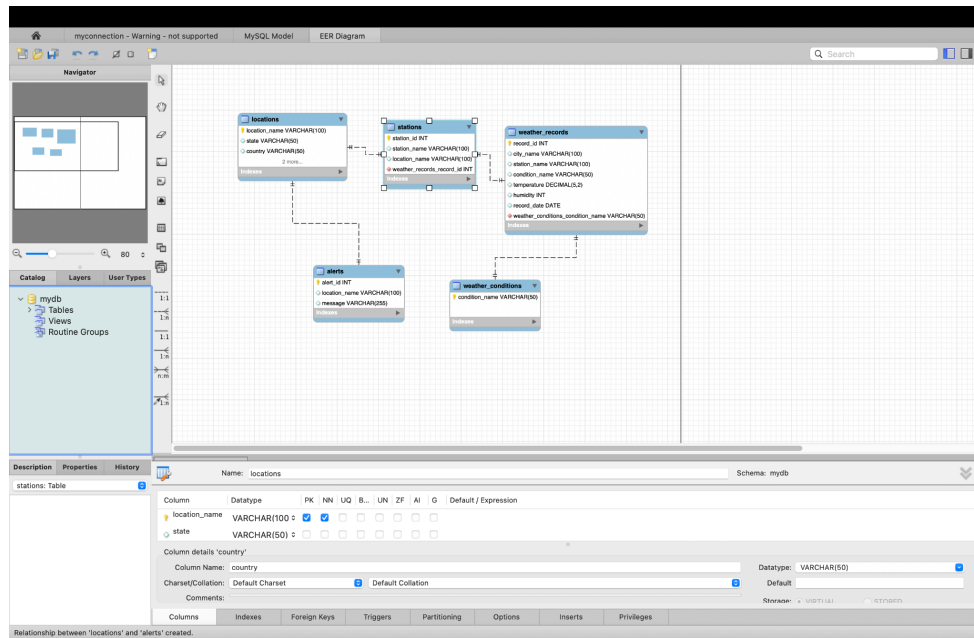
CSCE 548

Hardik Marlapudi

Spring 2026

Project 1:

Database Schema Diagram



```
PROBLEMS 68 DEBUG CONSOLE SONARQUBE 49 SPELL CHECKER 1 TERMINAL CODE REFERENCE LOG PORTS
hardikmarlapudi@Hardiks-MacBook-Pro CSE548-Project1 % mysql -u root

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 121
Server version: 9.6.0 Homebrew

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

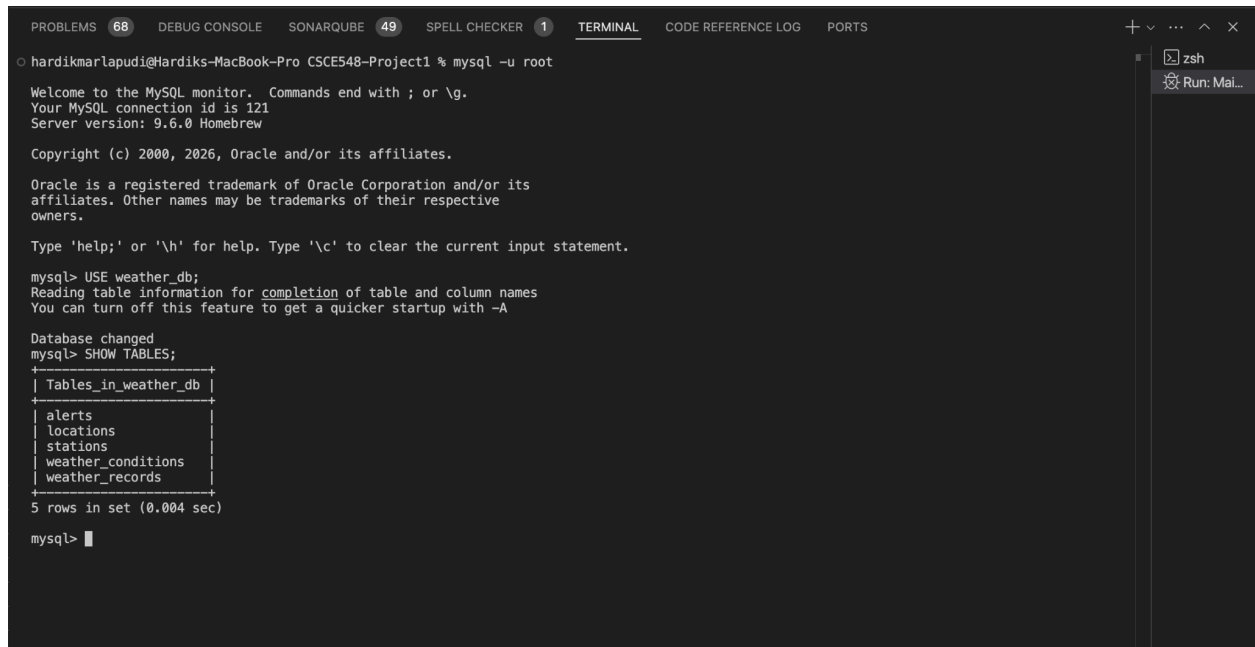
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE weather_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_weather_db |
+-----+
| alerts                |
| locations              |
| stations              |
| weather_conditions    |
| weather_records       |
+-----+
5 rows in set (0.004 sec)

mysql>
```

Tables Created



The screenshot shows a terminal window with a dark background. At the top, there is a navigation bar with tabs: PROBLEMS (68), DEBUG CONSOLE, SONARQUBE (49), SPELL CHECKER (1), TERMINAL (selected), CODE REFERENCE LOG, and PORTS. The terminal content shows a user logging into MySQL as root. The MySQL welcome message is displayed, followed by the user entering 'USE weather_db;'. The output shows the database has changed. Then, the user enters 'SHOW TABLES;', and the output displays a list of tables in the 'weather_db' database: alerts, locations, stations, weather_conditions, and weather_records. The output is formatted as a table with a header row and five data rows. The terminal prompt 'mysql>' is visible at the bottom.

```
hardikmarlapudi@Hardiks-MacBook-Pro CSCE548-Project1 % mysql -u root

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 121
Server version: 9.6.0 Homebrew

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

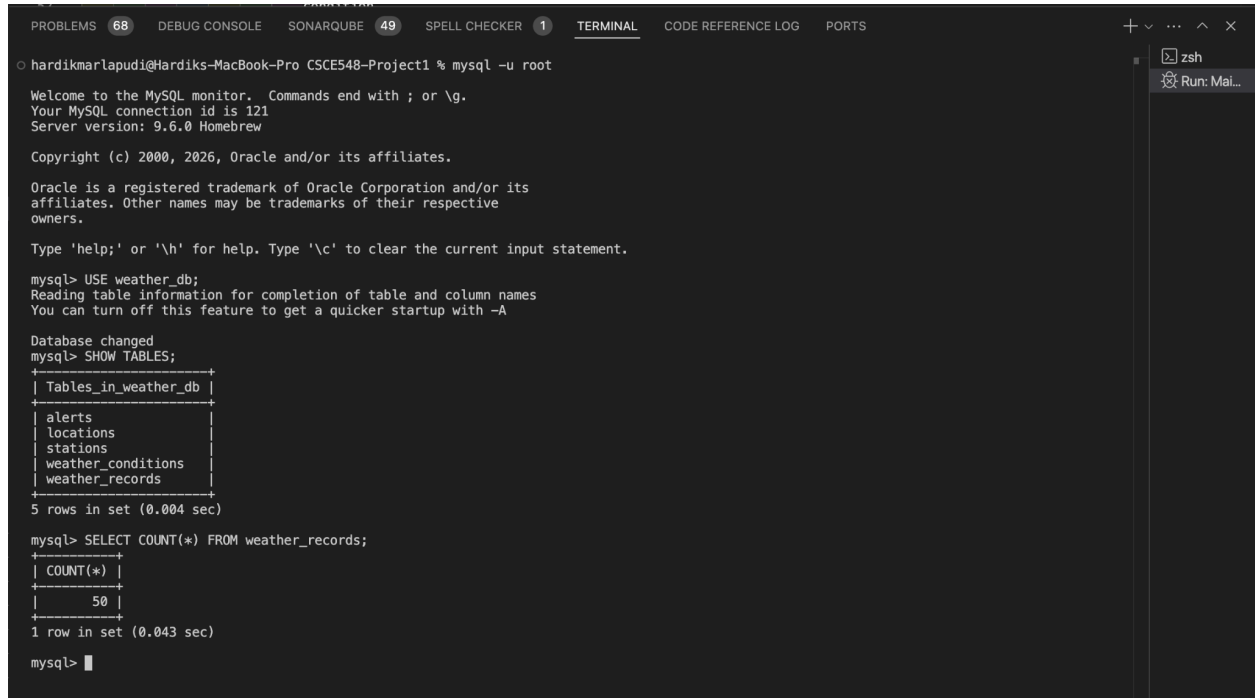
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE weather_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_weather_db |
+-----+
| alerts                |
| locations             |
| stations              |
| weather_conditions    |
| weather_records       |
+-----+
5 rows in set (0.004 sec)

mysql>
```

50+ Rows Verification



The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of a MySQL command prompt session. The user has connected to the MySQL server as root. The terminal shows the MySQL welcome message, the server version (9.6.0 Homebrew), and the database changed to weather_db. The user has executed the command 'SHOW TABLES;' and the output shows five tables: alerts, locations, stations, weather_conditions, and weather_records. The user has then executed the command 'SELECT COUNT(*) FROM weather_records;' and the output shows a single row with the count 50.

```
PROBLEMS 68 DEBUG CONSOLE SONARQUBE 49 SPELL CHECKER 1 TERMINAL CODE REFERENCE LOG PORTS
hardikmarlapudi@Hardiks-MacBook-Pro CSCE548-Project1 % mysql -u root

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 121
Server version: 9.6.0 Homebrew

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

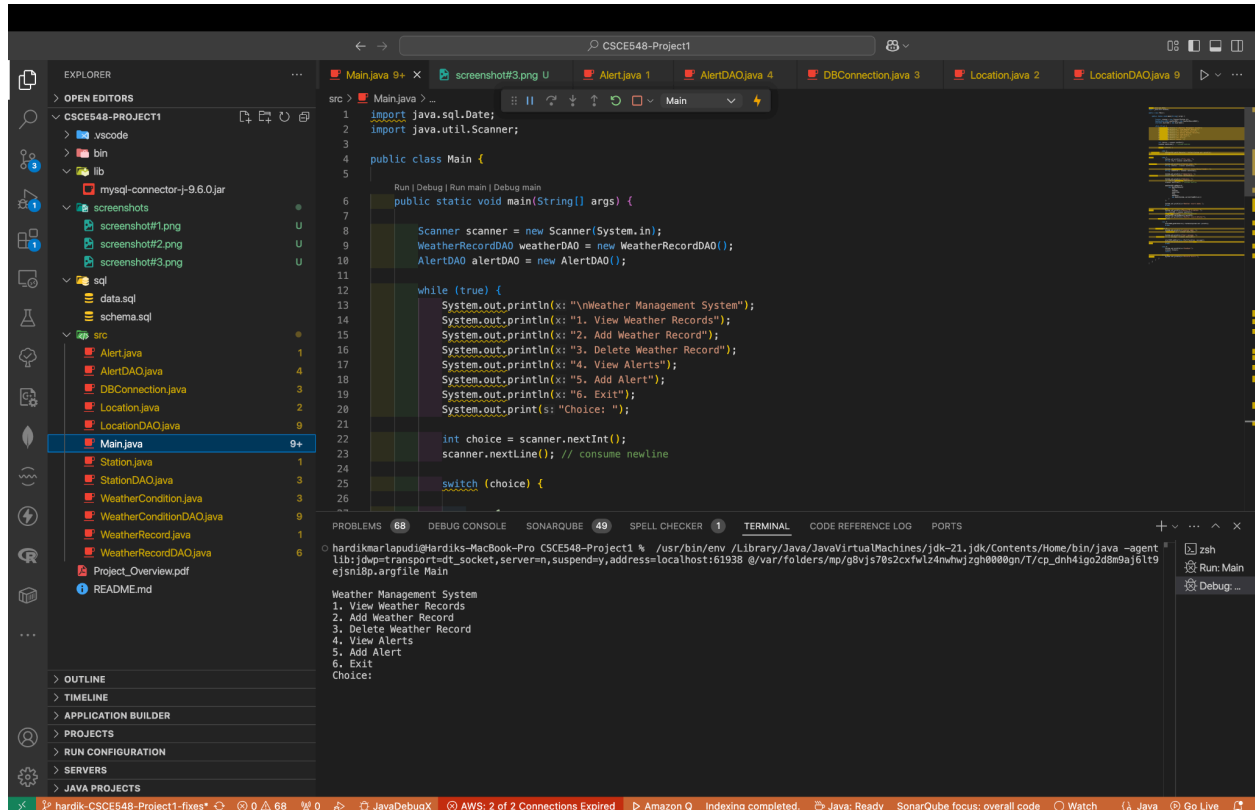
mysql> USE weather_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_weather_db |
+-----+
| alerts                |
| locations              |
| stations               |
| weather_conditions     |
| weather_records        |
+-----+
5 rows in set (0.004 sec)

mysql> SELECT COUNT(*) FROM weather_records;
+-----+
| COUNT(*) |
+-----+
|        50 |
+-----+
1 row in set (0.043 sec)

mysql>
```

Console Application Running



Data Retrieved from Database

The screenshot displays a code editor interface for a project named 'CSC548-Project1'. The Explorer panel on the left shows the project structure, including a 'src' directory with files like 'Alert.java', 'AlertDAO.java', 'DBConnection.java', 'Location.java', 'LocationDAO.java', 'Main.java', 'Station.java', 'StationDAO.java', 'WeatherConditionDAO.java', 'WeatherRecord.java', and 'WeatherRecordDAO.java'. The main editor area shows the 'data.sql' file with SQL queries for inserting weather data and alerts. The terminal at the bottom displays the output of the application, showing a list of weather records and alerts.

```
sql > data.sql
81 ('New York','Central Park Station','Rainy',39.1,84,'2025-01-21'),
82 ('New York','Central Park Station','Cloudy',42.3,63,'2025-01-22'),
83 ('New York','Central Park Station','Sunny',45.0,51,'2025-01-23'),
84 ('New York','Times Square Station','Rainy',40.2,85,'2025-01-24'),
85 ('New York','Times Square Station','Cloudy',43.1,62,'2025-01-25');
86
87 -- =====
88 -- ALERTS
89 -- =====
90 INSERT INTO alerts (location_name, message) VALUES
91 ('Columbia', 'Heavy rain expected'),
92 ('New York', 'Cold weather advisory');
93
```

PROBLEMS (27) DEBUG CONSOLE SONARQUBE (8) SPELL CHECKER (1) TERMINAL CODE REFERENCE LOG PORTS

45		City:	New York		Station:	Times Square Station		Condition:	Sunny		Temp:	44.2		Humidity:	52		Date:	2025-01-20
46		City:	New York		Station:	Central Park Station		Condition:	Rainy		Temp:	39.1		Humidity:	84		Date:	2025-01-21
47		City:	New York		Station:	Central Park Station		Condition:	Cloudy		Temp:	42.3		Humidity:	63		Date:	2025-01-22
48		City:	New York		Station:	Central Park Station		Condition:	Sunny		Temp:	45.0		Humidity:	51		Date:	2025-01-23
49		City:	New York		Station:	Times Square Station		Condition:	Rainy		Temp:	40.2		Humidity:	85		Date:	2025-01-24
50		City:	New York		Station:	Times Square Station		Condition:	Cloudy		Temp:	43.1		Humidity:	62		Date:	2025-01-25

Weather Management System
1. View Weather Records
2. Add Weather Record
3. Delete Weather Record
4. View Alerts
5. Add Alert
6. Exit
Choice: 4
1 | Location: Columbia | Message: Heavy rain expected
2 | Location: New York | Message: Cold weather advisory

Weather Management System
1. View Weather Records
2. Add Weather Record
3. Delete Weather Record
4. View Alerts
5. Add Alert
6. Exit
Choice: 1

Write the Overview Document Section

This project is based on modeling a Weather Management System, which stores and manages weather details based on location and station. The system monitors daily weather conditions such as temperature and humidity levels, and it is capable of generating location-based weather alerts. The application is based on the integration of a database with a Java-based console interface.

AI Prompt Used

ChatGPT was employed for creating the initial SQL schema, test data, and Java Data Access Object (DAO) code. The prompts were centered on creating a relational database with various tables, establishing foreign key relationships between them, and creating CRUD operations for the project. The result obtained from using ChatGPT was a starting point, which was later verified and corrected using manual debugging and testing.

Changes I Made Manually

Example:

- Removed unnecessary join tables generated by AI
- Replaced users table with stations to better fit the problem
- Fixed SQL column mismatches causing runtime errors
- Corrected DAO queries to match final schema
- Ensured 50 sequential weather records
- Debugged MySQL connection and missing-table errors

AI Effectiveness Analysis

ChatGPT helped hasten the development process, but it created overly complex schemas and used columns inconsistently. Debugging and refactoring were necessary to correct it and ensure compliance with foreign keys.

Project 2:

- Project 2 extends Project 1.
- Introduces Business Layer and Service Layer.
- Converts the application into a microservice-based architecture.

System Architecture

Layered Architecture Design

Bullet Points:

- Data Layer (DAO Classes)
- Business Layer
- Service Layer (Microservices)
- Console Client
- MySQL Database Backend

- The DAO layer handles database interaction.
- The business layer contains validation logic.
- Service layer exposes REST-style endpoints.
- Console client consumes the services.
- Fully separated architecture.

Services Hosted

Title: Microservices Hosted

Bullet points:

- Weather Service: <http://localhost:8081/weather>
- Alert Service: <http://localhost:8082/alerts>
- Location Service: <http://localhost:8083/locations>
- Each service runs independently
- Hosted using Java HttpServer

They Each:

- Each service runs on a different port.
- They invoke the business layer.
- Demonstrates service-oriented architecture.

Console Client Testing

Console Client Testing Services:

- Client calls each service endpoint
- Displays returned data
- Confirms database connectivity
- Demonstrates integration of all layers
- Console client acts as front-end
- Verifies microservices return correct data
- Shows full system integration

Database Verification

- Verified 50+ weather records
- Ensured foreign key relationships
- Confirmed data integrity
- Validated MySQL connection

CRUD Operations

CRUD Functionality Implemented

- Create weather records
- Retrieve weather data
- Update alerts and locations
- Delete records
- All operations are handled via the DAO layer
- DAO classes implement CRUD.
- Business layer validates.
- Services expose read functionality.

AI Usage

AI Usage in Development

- Generated initial SQL schema
- Generated DAO classes
- Assisted with service structure
- Required debugging and refinement
- Improved development speed

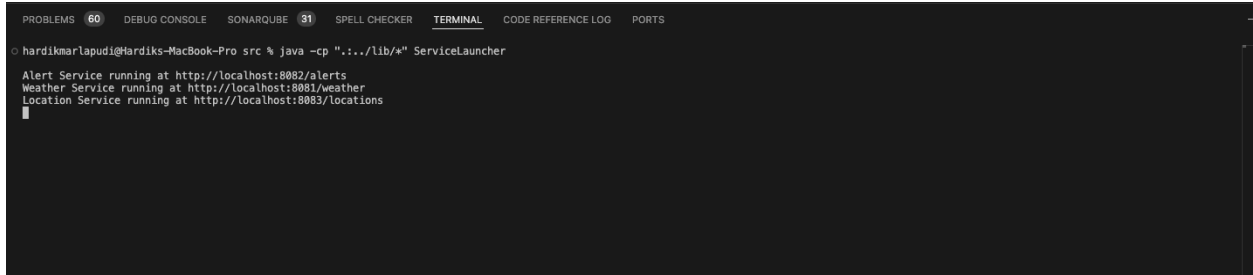
Manual Changes Made

- Simplified overly complex AI schema
- Corrected foreign key mismatches
- Fixed MySQL driver classpath issues
- Debugged connection errors
- Adjusted service endpoints

AI Effectiveness Analysis

- Accelerated development process
- Provided a structural starting point
- Required manual validation
- Produced inconsistencies in the schema
- Helpful but not fully reliable without debugging

Screenshot #1



```
PROBLEMS 60 DEBUG CONSOLE SONARQUBE 31 SPELL CHECKER TERMINAL CODE REFERENCE LOG PORTS
hardikmarlapudi@Hardiks-MacBook-Pro src % java -cp "..." ServiceLauncher
Alert Service running at http://localhost:8082/alerts
Weather Service running at http://localhost:8081/weather
Location Service running at http://localhost:8083/locations
```

Screenshot #1 — Summary

The screenshot shows a terminal where a Java application (**ServiceLauncher**) has started successfully.

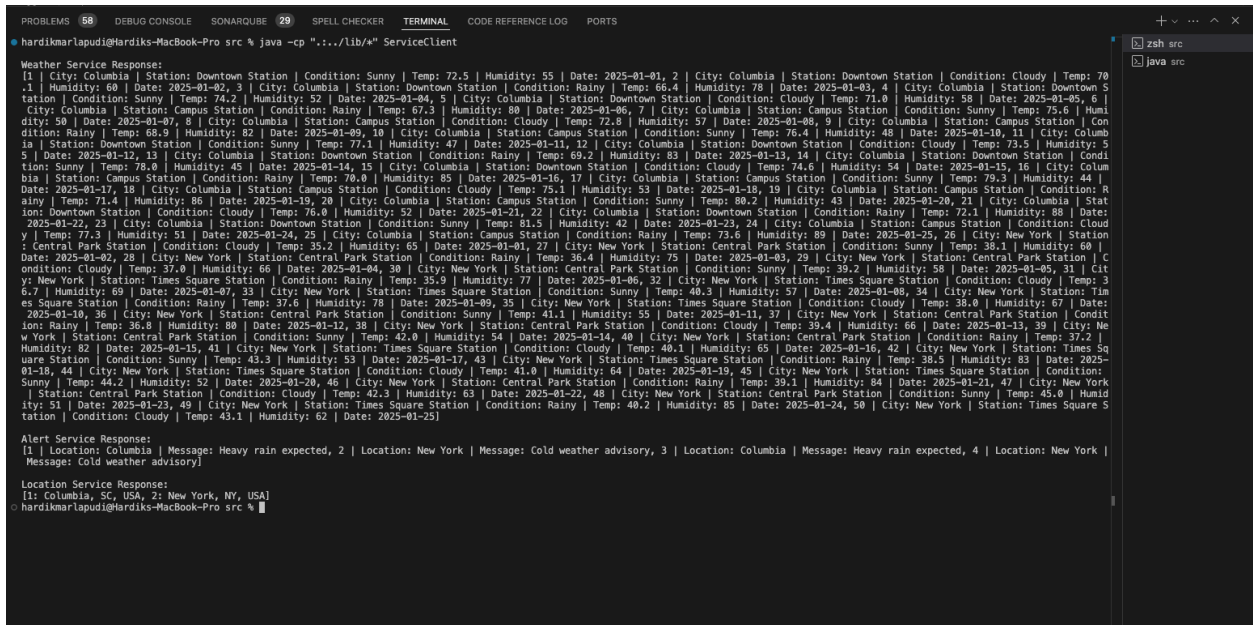
When the program runs, it launches three backend services on your computer:

- Weather Service → <http://localhost:8081/weather>
- Alert Service → <http://localhost:8082/alerts>
- Location Service → <http://localhost:8083/locations>

Each service runs on a different port number and waits for requests.

Since the terminal is still open and not showing errors, it means all services are running correctly and ready to use.

Screenshot #2



```
PROBLEMS 58 DEBUG CONSOLE SONARQUBE 29 SPELL CHECKER TERMINAL CODE REFERENCE LOG PORTS
hardikmarlapudi@hardiks-MacBook-Pro src % java -cp "..." Lib\4" ServiceClient

Weather Service Response:
[1 | City: Columbia | Station: Downtown Station | Condition: Sunny | Temp: 72.5 | Humidity: 55 | Date: 2025-01-01, 2 | City: Columbia | Station: Downtown Station | Condition: Cloudy | Temp: 70.1 | Humidity: 60 | Date: 2025-01-02, 3 | City: Columbia | Station: Downtown Station | Condition: Rainy | Temp: 66.4 | Humidity: 78 | Date: 2025-01-03, 4 | City: Columbia | Station: Downtown Station | Condition: Sunny | Temp: 74.2 | Humidity: 52 | Date: 2025-01-04, 5 | City: Columbia | Station: Downtown Station | Condition: Cloudy | Temp: 71.0 | Humidity: 58 | Date: 2025-01-05, 6 | City: Columbia | Station: Campus Station | Condition: Rainy | Temp: 67.3 | Humidity: 80 | Date: 2025-01-06, 7 | City: Columbia | Station: Campus Station | Condition: Sunny | Temp: 75.6 | Humidity: 58 | Date: 2025-01-07, 8 | City: Columbia | Station: Campus Station | Condition: Cloudy | Temp: 72.8 | Humidity: 57 | Date: 2025-01-08, 9 | City: Columbia | Station: Campus Station | Condition: Rainy | Temp: 68.9 | Humidity: 82 | Date: 2025-01-09, 10 | City: Columbia | Station: Campus Station | Condition: Sunny | Temp: 76.4 | Humidity: 48 | Date: 2025-01-10, 11 | City: Columbia | Station: Downtown Station | Condition: Sunny | Temp: 77.1 | Humidity: 47 | Date: 2025-01-11, 12 | City: Columbia | Station: Downtown Station | Condition: Cloudy | Temp: 73.5 | Humidity: 55 | Date: 2025-01-12, 13 | City: Columbia | Station: Downtown Station | Condition: Rainy | Temp: 69.2 | Humidity: 83 | Date: 2025-01-13, 14 | City: Columbia | Station: Downtown Station | Condition: Sunny | Temp: 78.0 | Humidity: 45 | Date: 2025-01-14, 15 | City: Columbia | Station: Downtown Station | Condition: Cloudy | Temp: 74.6 | Humidity: 54 | Date: 2025-01-15, 16 | City: Columbia | Station: Campus Station | Condition: Rainy | Temp: 78.0 | Humidity: 85 | Date: 2025-01-16, 17 | City: Columbia | Station: Campus Station | Condition: Sunny | Temp: 79.3 | Humidity: 44 | Date: 2025-01-17, 18 | City: Columbia | Station: Campus Station | Condition: Cloudy | Temp: 75.1 | Humidity: 53 | Date: 2025-01-18, 19 | City: Columbia | Station: Campus Station | Condition: Rainy | Temp: 71.4 | Humidity: 66 | Date: 2025-01-19, 20 | City: Columbia | Station: Campus Station | Condition: Sunny | Temp: 80.2 | Humidity: 43 | Date: 2025-01-20, 21 | City: Columbia | Station: Downtown Station | Condition: Cloudy | Temp: 76.0 | Humidity: 52 | Date: 2025-01-21, 22 | City: Columbia | Station: Downtown Station | Condition: Rainy | Temp: 72.1 | Humidity: 88 | Date: 2025-01-22, 23 | City: Columbia | Station: Downtown Station | Condition: Sunny | Temp: 81.5 | Humidity: 42 | Date: 2025-01-23, 24 | City: Columbia | Station: Campus Station | Condition: Cloudy | Temp: 77.3 | Humidity: 51 | Date: 2025-01-24, 25 | City: Columbia | Station: Campus Station | Condition: Rainy | Temp: 73.6 | Humidity: 69 | Date: 2025-01-25, 26 | City: New York | Station: Central Park Station | Condition: Cloudy | Temp: 35.2 | Humidity: 65 | Date: 2025-01-01, 27 | City: New York | Station: Central Park Station | Condition: Sunny | Temp: 38.1 | Humidity: 60 | Date: 2025-01-02, 28 | City: New York | Station: Central Park Station | Condition: Rainy | Temp: 36.4 | Humidity: 75 | Date: 2025-01-03, 29 | City: New York | Station: Central Park Station | Condition: Cloudy | Temp: 37.0 | Humidity: 66 | Date: 2025-01-04, 30 | City: New York | Station: Central Park Station | Condition: Sunny | Temp: 39.2 | Humidity: 58 | Date: 2025-01-05, 31 | City: New York | Station: Times Square Station | Condition: Rainy | Temp: 35.9 | Humidity: 77 | Date: 2025-01-06, 32 | City: New York | Station: Times Square Station | Condition: Cloudy | Temp: 36.7 | Humidity: 69 | Date: 2025-01-07, 33 | City: New York | Station: Times Square Station | Condition: Sunny | Temp: 40.3 | Humidity: 57 | Date: 2025-01-08, 34 | City: New York | Station: Times Square Station | Condition: Rainy | Temp: 37.6 | Humidity: 78 | Date: 2025-01-09, 35 | City: New York | Station: Times Square Station | Condition: Cloudy | Temp: 38.0 | Humidity: 67 | Date: 2025-01-10, 36 | City: New York | Station: Central Park Station | Condition: Sunny | Temp: 41.2 | Humidity: 55 | Date: 2025-01-11, 37 | City: New York | Station: Central Park Station | Condition: Rainy | Temp: 36.8 | Humidity: 80 | Date: 2025-01-12, 38 | City: New York | Station: Central Park Station | Condition: Cloudy | Temp: 39.4 | Humidity: 66 | Date: 2025-01-13, 39 | City: New York | Station: Central Park Station | Condition: Sunny | Temp: 42.0 | Humidity: 54 | Date: 2025-01-14, 40 | City: New York | Station: Central Park Station | Condition: Rainy | Temp: 37.2 | Humidity: 82 | Date: 2025-01-15, 41 | City: New York | Station: Times Square Station | Condition: Cloudy | Temp: 40.1 | Humidity: 65 | Date: 2025-01-16, 42 | City: New York | Station: Times Square Station | Condition: Sunny | Temp: 43.3 | Humidity: 53 | Date: 2025-01-17, 43 | City: New York | Station: Times Square Station | Condition: Rainy | Temp: 38.5 | Humidity: 83 | Date: 2025-01-18, 44 | City: New York | Station: Times Square Station | Condition: Cloudy | Temp: 41.0 | Humidity: 64 | Date: 2025-01-19, 45 | City: New York | Station: Times Square Station | Condition: Sunny | Temp: 44.2 | Humidity: 52 | Date: 2025-01-20, 46 | City: New York | Station: Central Park Station | Condition: Rainy | Temp: 39.1 | Humidity: 84 | Date: 2025-01-21, 47 | City: New York | Station: Central Park Station | Condition: Cloudy | Temp: 42.2 | Humidity: 63 | Date: 2025-01-22, 48 | City: New York | Station: Central Park Station | Condition: Sunny | Temp: 45.0 | Humidity: 51 | Date: 2025-01-23, 49 | City: New York | Station: Times Square Station | Condition: Rainy | Temp: 40.2 | Humidity: 85 | Date: 2025-01-24, 50 | City: New York | Station: Times Square Station | Condition: Cloudy | Temp: 43.1 | Humidity: 62 | Date: 2025-01-25]

Alert Service Response:
[1 | Location: Columbia | Message: Heavy rain expected, 2 | Location: New York | Message: Cold weather advisory, 3 | Location: Columbia | Message: Heavy rain expected, 4 | Location: New York | Message: Cold weather advisory]

Location Service Response:
[1: Columbia, SC, USA, 2: New York, NY, USA]
hardikmarlapudi@hardiks-MacBook-Pro src %
```

This screenshot shows the ServiceClient program running in the terminal and successfully communicating with the backend services started earlier.

The client sends requests to the services and prints their responses:

Weather Service Response

Displays weather data for cities (e.g., Columbia and New York), including:

- station name
- weather condition

- **temperature**
- **humidity**
- **date**

Alert Service Response

Shows alert messages such as:

- **heavy rain expected**
- **cold weather advisory**

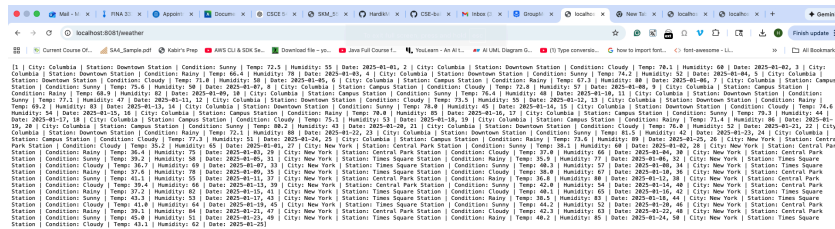
Location Service Response

Returns location information:

- **Columbia, SC, USA**
- **New York, NY, USA**

Overall, this confirms that the client application successfully connects to all three services and retrieves data correctly, meaning the system is working end-to-end.

Screenshot #3

The screenshot shows a web browser window with the address bar displaying 'localhost:8081/weather'. The page content is a long, dense list of weather records. Each record contains the following information: City name (e.g., Columbia, New York), Station name (e.g., Downtown Station, Times Square Station), Weather condition (e.g., Sunny, Cloudy, Rainy), Temperature (Temp), Humidity (Humidity), and Date (Date). The records are listed in a single column, separated by line breaks. The browser's address bar and tabs are visible at the top of the window.

This screenshot shows the Weather Service API running in a web browser at:

<http://localhost:8081/weather>

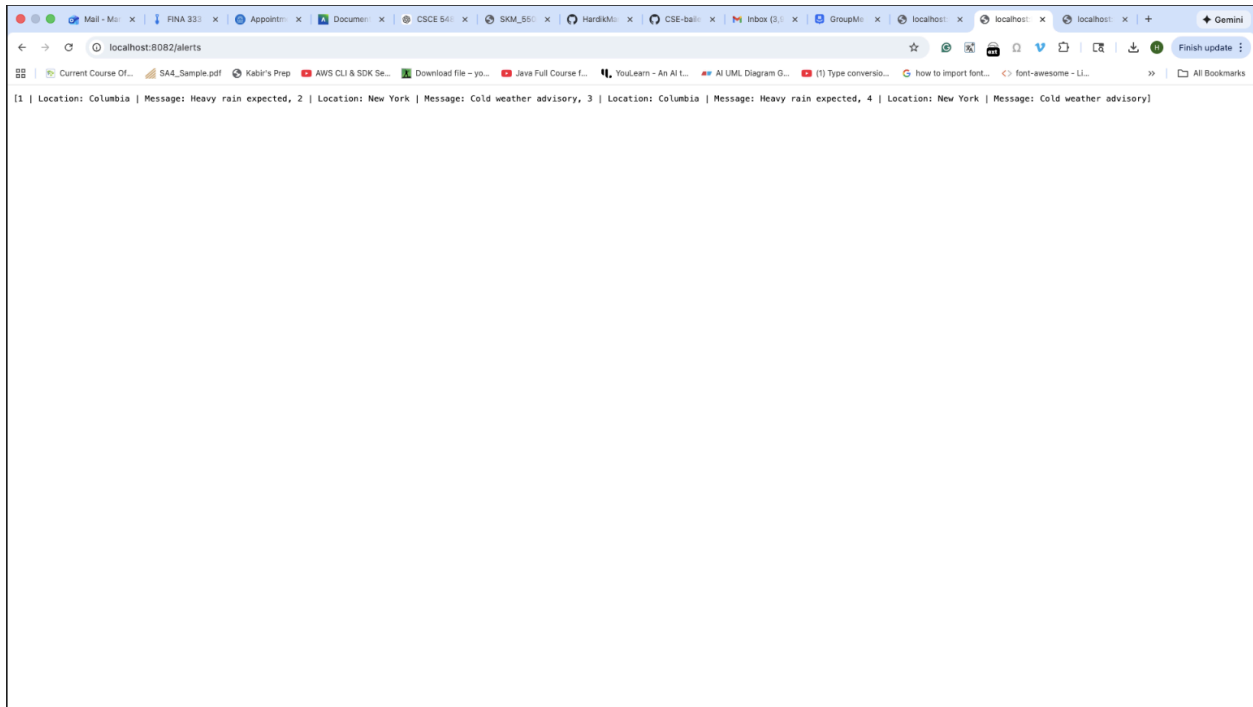
The page displays weather data returned directly from the Weather Service.

☒ It lists multiple weather records including:

- City name (Columbia, New York)
- Weather station
- Weather condition (Sunny, Cloudy, Rainy)
- Temperature
- Humidity
- Date

This confirms that the Weather Service backend is working correctly and the browser successfully retrieved live data from the local server.

Screenshot #4



This screenshot shows the Alert Service API running in a web browser at:

`http://localhost:8082/alerts`

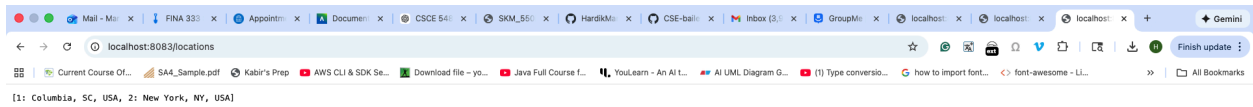
The page displays alert messages returned by the Alert Service, including:

- Location (Columbia, New York)
- Alert messages such as Heavy rain expected and Cold weather advisory

This confirms that the Alert Service backend is running successfully and responding to browser requests with alert data.

In short: the alerts microservice is working correctly and serving data through its API endpoint.

Screenshot #5



This screenshot shows the Location Service API running in a web browser at:

<http://localhost:8083/locations>

The page displays location data returned by the service:

- Columbia, SC, USA
- New York, NY, USA

This confirms that the Location Service is running successfully and responding to HTTP requests.

In short: the location microservice is working correctly and providing location information through its API endpoint.

Conclusion

Project 2 Summary

- Implemented layered architecture
- Hosted three microservices
- Integrated database with services
- Built console client
- Successfully demonstrated full system functionality

CSCE 548 - Project 3:

Screenshot #1:

ID	City	State	Country	Temperature	Humidity	Date
1	San Jose	CA	USA	68°F	55%	2023-10-26
2	San Jose	CA	USA	68°F	55%	2023-10-26
3	Chicago	IL	USA	57°F	65%	2023-10-26
4	Chicago	IL	USA	57°F	65%	2023-10-26
5	Phoenix	AZ	USA	80°F	40%	2023-10-26
6	Phoenix	AZ	USA	80°F	40%	2023-10-26
7	Philadelphia	PA	USA	71°F	60%	2023-10-26
8	Philadelphia	PA	USA	71°F	60%	2023-10-26
9	San Diego	CA	USA	68°F	55%	2023-10-26
10	San Diego	CA	USA	68°F	55%	2023-10-26
11	San Jose	CA	USA	68°F	55%	2023-10-26
12	San Jose	CA	USA	68°F	55%	2023-10-26
13	San Jose	CA	USA	68°F	55%	2023-10-26
14	San Jose	CA	USA	68°F	55%	2023-10-26
15	San Jose	CA	USA	68°F	55%	2023-10-26
16	San Jose	CA	USA	68°F	55%	2023-10-26
17	San Jose	CA	USA	68°F	55%	2023-10-26
18	San Jose	CA	USA	68°F	55%	2023-10-26
19	San Jose	CA	USA	68°F	55%	2023-10-26
20	San Jose	CA	USA	68°F	55%	2023-10-26
21	San Jose	CA	USA	68°F	55%	2023-10-26
22	San Jose	CA	USA	68°F	55%	2023-10-26
23	San Jose	CA	USA	68°F	55%	2023-10-26
24	San Jose	CA	USA	68°F	55%	2023-10-26
25	San Jose	CA	USA	68°F	55%	2023-10-26
26	San Jose	CA	USA	68°F	55%	2023-10-26
27	San Jose	CA	USA	68°F	55%	2023-10-26
28	San Jose	CA	USA	68°F	55%	2023-10-26
29	San Jose	CA	USA	68°F	55%	2023-10-26
30	San Jose	CA	USA	68°F	55%	2023-10-26
31	San Jose	CA	USA	68°F	55%	2023-10-26
32	San Jose	CA	USA	68°F	55%	2023-10-26
33	San Jose	CA	USA	68°F	55%	2023-10-26
34	San Jose	CA	USA	68°F	55%	2023-10-26
35	San Jose	CA	USA	68°F	55%	2023-10-26
36	San Jose	CA	USA	68°F	55%	2023-10-26
37	San Jose	CA	USA	68°F	55%	2023-10-26
38	San Jose	CA	USA	68°F	55%	2023-10-26
39	San Jose	CA	USA	68°F	55%	2023-10-26
40	San Jose	CA	USA	68°F	55%	2023-10-26
41	San Jose	CA	USA	68°F	55%	2023-10-26
42	San Jose	CA	USA	68°F	55%	2023-10-26
43	San Jose	CA	USA	68°F	55%	2023-10-26
44	San Jose	CA	USA	68°F	55%	2023-10-26
45	San Jose	CA	USA	68°F	55%	2023-10-26
46	San Jose	CA	USA	68°F	55%	2023-10-26
47	San Jose	CA	USA	68°F	55%	2023-10-26
48	San Jose	CA	USA	68°F	55%	2023-10-26
49	San Jose	CA	USA	68°F	55%	2023-10-26
50	San Jose	CA	USA	68°F	55%	2023-10-26

Range Weather Records

Start Date: [] End Date: [] Search [] Refresh []

© 2024 CSCE 548 Project 3 - Weather Application

Summary: The output demonstrates that the Weather microservice is fully operational. The frontend successfully sends a request to the backend, which retrieves 50 records from the MySQL database and returns them to be displayed on the page. This confirms that your REST API, DAO layer, database connectivity, and frontend integration are all working correctly.

Screenshot #2:

ID	City	State	Weather	Temperature	Humidity	Date
1	New York	NY	Sunny	80°F	65%	2023-01-01
2	Los Angeles	CA	Cloudy	70°F	70%	2023-01-02
3	Chicago	IL	Rainy	60°F	80%	2023-01-03
4	Phoenix	AZ	Sunny	90°F	55%	2023-01-04
5	Philadelphia	PA	Cloudy	65°F	75%	2023-01-05
6	San Antonio	TX	Sunny	85°F	60%	2023-01-06
7	San Diego	CA	Cloudy	75°F	70%	2023-01-07
8	Dallas	TX	Sunny	95°F	50%	2023-01-08
9	San Jose	CA	Rainy	60°F	85%	2023-01-09
10	Seattle	WA	Cloudy	70°F	80%	2023-01-10
11	San Francisco	CA	Sunny	80°F	70%	2023-01-11
12	Portland	OR	Rainy	65°F	90%	2023-01-12
13	Denver	CO	Sunny	90°F	55%	2023-01-13
14	San Francisco	CA	Cloudy	75°F	75%	2023-01-14
15	Seattle	WA	Rainy	60°F	85%	2023-01-15
16	Phoenix	AZ	Sunny	95°F	50%	2023-01-16
17	San Antonio	TX	Sunny	85°F	60%	2023-01-17
18	San Diego	CA	Cloudy	75°F	70%	2023-01-18
19	Dallas	TX	Sunny	95°F	50%	2023-01-19
20	San Jose	CA	Rainy	60°F	85%	2023-01-20
21	Seattle	WA	Cloudy	70°F	80%	2023-01-21
22	San Francisco	CA	Sunny	80°F	70%	2023-01-22
23	Portland	OR	Rainy	65°F	90%	2023-01-23
24	Denver	CO	Sunny	90°F	55%	2023-01-24
25	San Francisco	CA	Cloudy	75°F	75%	2023-01-25

Manage Weather Records

ID: Name: Save

Summary: The screenshot shows a Weather Application dashboard running locally in the browser (127.0.0.1:5500). The page displays a Weather Records Table that lists weather data retrieved from a backend microservice connected to a MySQL database. Each record includes the ID, City, State, Weather Condition, Temperature, Humidity, and Date.

At the bottom of the page, there is a “Manage Weather Records” form that allows users to perform CRUD operations (Add, Update, Delete) on the weather records. The highlighted section indicates the latest record displayed in the table and the input fields used to manage records. Overall, the screenshot demonstrates that the frontend interface, backend service, and database integration are functioning correctly for managing weather data.

Screenshot #3:

Weather Application
CSCE 548 - Building Secure Software

[Load Weather \(UI\)](#) [Load Weather \(JSON\)](#)

Weather Records Table

ID	City	State	Condition	Temperature	Humidity	Date
1	New York	NY	Rainy	66.4°F	78%	2025-01-03
2	Los Angeles	CA	Sunny	38.1°F	60%	2025-01-04
3	Chicago	IL	Cloudy	37°F	66%	2025-01-05
4	Houston	TX	Rainy	36.4°F	75%	2025-01-06
5	Phoenix	AZ	Cloudy	60°F	44%	2025-02-02
6	Philadelphia	PA	Rainy	71°F	70%	2025-02-03
7	San Antonio	TX	Sunny	87°F	76%	2025-02-04
8	San Diego	CA	Cloudy	86°F	52%	2025-02-05
9	Dallas	TX	Rainy	78°F	59%	2025-02-06
10	San Jose	CA	Sunny	61°F	45%	2025-02-07
11	Austin	TX	Cloudy	71°F	66%	2025-02-08
12	Jacksonville	FL	Rainy	77°F	53%	2025-02-09
13	Fort Worth	TX	Sunny	79°F	59%	2025-02-10

Weather JSON:

```
[{"id": 1, "cityName": "New York", "stateName": "NY", "conditionName": "Rainy", "temp": 66.4, "humidity": 78, "date": "2025-01-03"}, {"id": 2, "cityName": "Los Angeles", "stateName": "CA", "conditionName": "Sunny", "temp": 38.1, "humidity": 60, "date": "2025-01-04"}, {"id": 3, "cityName": "Chicago", "stateName": "IL", "conditionName": "Cloudy", "temp": 37, "humidity": 66, "date": "2025-01-05"}, {"id": 4, "cityName": "Houston", "stateName": "TX", "conditionName": "Rainy", "temp": 36.4, "humidity": 75, "date": "2025-01-06"}, {"id": 5, "cityName": "Phoenix", "stateName": "AZ", "conditionName": "Cloudy", "temp": 60, "humidity": 44, "date": "2025-02-02"}, {"id": 6, "cityName": "Philadelphia", "stateName": "PA", "conditionName": "Rainy", "temp": 71, "humidity": 70, "date": "2025-02-03"}, {"id": 7, "cityName": "San Antonio", "stateName": "TX", "conditionName": "Sunny", "temp": 87, "humidity": 76, "date": "2025-02-04"}, {"id": 8, "cityName": "San Diego", "stateName": "CA", "conditionName": "Cloudy", "temp": 86, "humidity": 52, "date": "2025-02-05"}, {"id": 9, "cityName": "Dallas", "stateName": "TX", "conditionName": "Rainy", "temp": 78, "humidity": 59, "date": "2025-02-06"}, {"id": 10, "cityName": "San Jose", "stateName": "CA", "conditionName": "Sunny", "temp": 61, "humidity": 45, "date": "2025-02-07"}, {"id": 11, "cityName": "Austin", "stateName": "TX", "conditionName": "Cloudy", "temp": 71, "humidity": 66, "date": "2025-02-08"}, {"id": 12, "cityName": "Jacksonville", "stateName": "FL", "conditionName": "Rainy", "temp": 77, "humidity": 53, "date": "2025-02-09"}, {"id": 13, "cityName": "Fort Worth", "stateName": "TX", "conditionName": "Sunny", "temp": 79, "humidity": 59, "date": "2025-02-10"}]
```

Summary: This output shows the Location Microservice successfully retrieving and displaying location data from the database. When the “Load Locations” button was clicked, the frontend sent a GET request to the Location Service (running on port 8083). The service accessed the **locations** table in the MySQL database and returned the stored location records.

Conclusion

Project 3 summary

The application was structured using a multi-layered architecture that included:

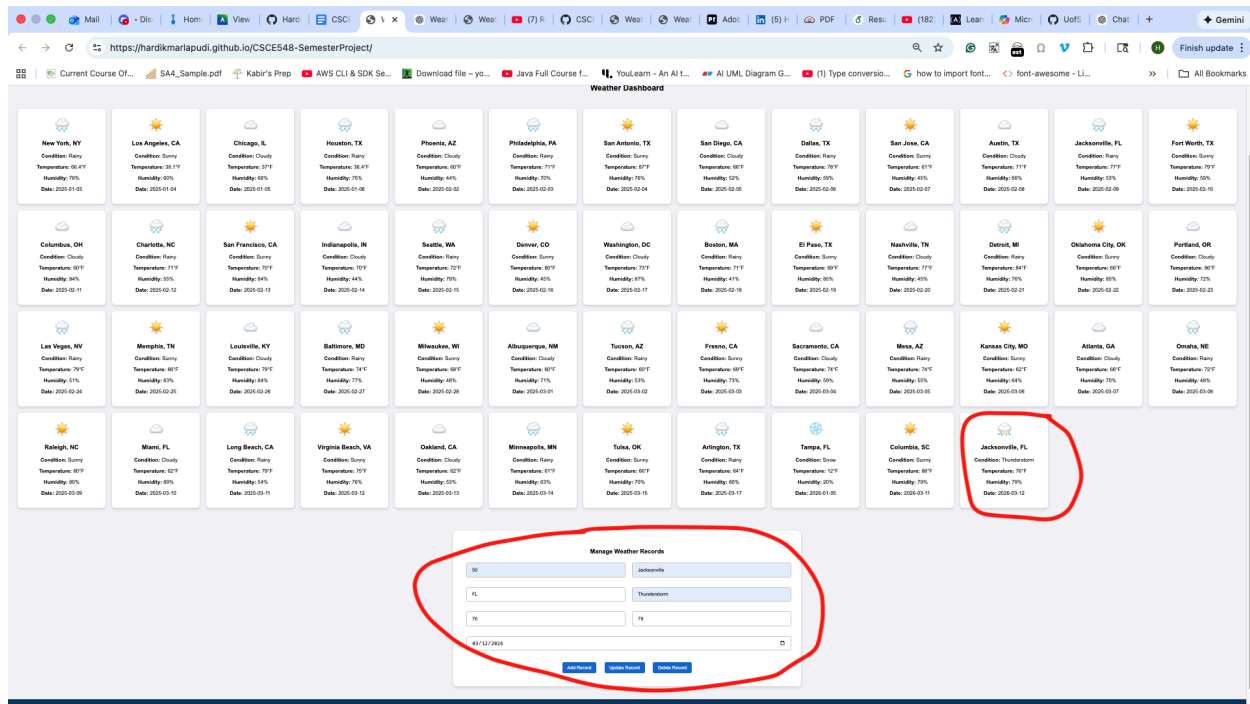
- **Presentation Layer (HTML, JavaScript frontend)**
- **Service Layer (REST microservices using Java HttpServer)**
- **Business Layer (validation and logic handling)**
- **DAO Layer (database access using JDBC)**
- **MySQL Database (data persistence)**

All services successfully connect to a MySQL database and return real-time data, demonstrating full-stack integration and secure microservice architecture.

Overall, Project 3 demonstrates proper RESTful design principles, database integration, secure coding practices, and modular software architecture suitable for scalable backend systems.

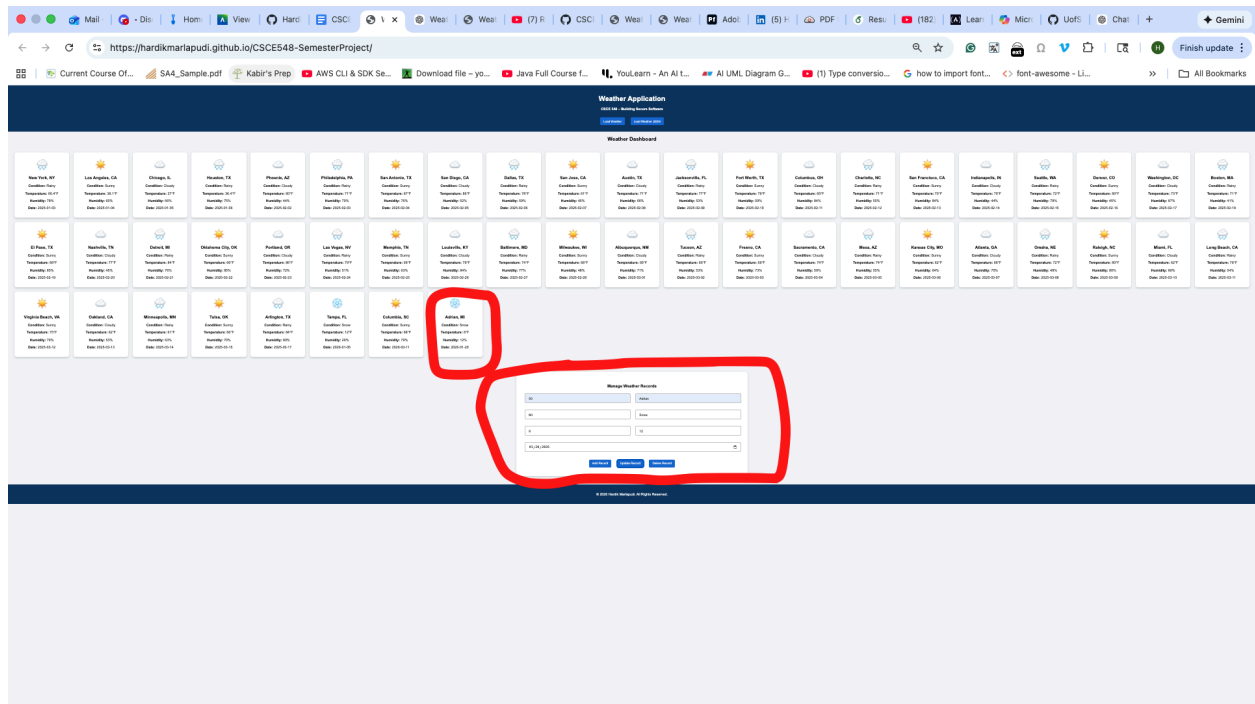
CSCE 548 - Project 4:

Screenshot #1 - ADD Record (POST)



Summary: The Add Record feature allows users to insert new weather data into the system by entering information such as the city, state, weather condition, temperature, humidity, and date. When the user submits the form, a POST request is sent to the Weather microservice API. The backend processes the request and stores the new record in the MySQL database.

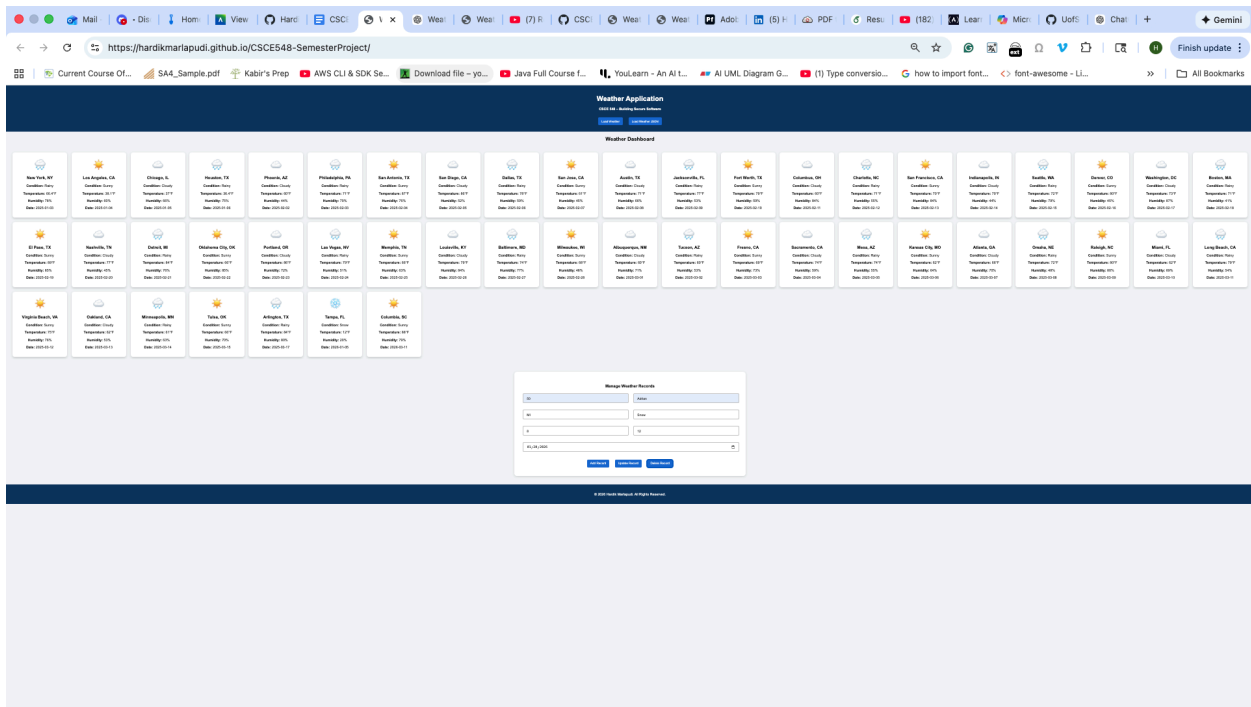
Screenshot #2 - EDIT Record (PUT)



Summary: The Edit Record feature allows users to modify existing weather records.

The user enters the record ID and updated weather information. The application then sends a PUT request to the Weather service endpoint. The backend updates the corresponding record in the database using prepared SQL statements to ensure secure data handling.

Screenshot #3 - DELETE Record (DELETE)



Note: The weather card with an ID number of 50 has been deleted.

Summary: The Delete Record feature enables users to remove weather records from the database. When a record ID is entered and the delete button is clicked, a DELETE request is sent to the Weather microservice. The service then removes the specified record from the database. The user interface automatically refreshes to display the updated weather data.

Conclusion

Project 4 summary: Project 4 expanded the Weather Application by integrating full database interaction and REST API functionality. The project connects a frontend user interface with multiple backend microservices, including the Weather Service, Alert Service, and Location Service.

The application allows users to retrieve weather data in two formats: JSON output for raw API responses and visual weather cards for a user-friendly interface. Users can also interact with the system by adding, updating, and deleting weather records through the implemented CRUD operations.

The backend services were developed using Java microservices with an HTTP server, while the frontend was built using HTML, CSS, and JavaScript. The services communicate with a MySQL database, which stores weather records and related information. Secure database access was implemented using prepared SQL statements to help prevent SQL injection and improve system reliability.

Overall, this project demonstrates how microservices architecture, RESTful APIs, frontend interfaces, and database systems work together to build a secure and interactive software application. The implementation of CRUD operations shows how users can actively manage data while maintaining synchronization between the user interface and the backend services.